
LANGCHAIN vs LANGGRAPH

vs a Hand-Rolled Pipeline -- Case Study: Qcompass-Chatbot-KPI

CHAINS | STATEGRAPH | DICT ROUTING | RETRY LOOPS | TRADE-OFFS

- 1 What is LangChain?
- 2 What is LangGraph?
- 3 What Qcompass-Chatbot-KPI Actually Does
- 4 Deep Dive: Dict-Based Routing as 'Graph Routing'
- 5 The Difference, Concretely
- 6 Pros and Cons
- 7 Master Summary Table
- 8 Decision Guide
- 9 Interview Cheat Sheet (Q&A;)
- 10 One-Page Quick Reference

1 What is LangChain?

LangChain is a Python/JS framework providing pre-built abstractions for LLM apps: prompt templates, output parsers, chains (sequences of LLM calls + logic), retrievers, vector store integrations, memory classes, and agents.

Core building blocks:

- LLM / ChatModel wrappers -- unified interface over OpenAI, Anthropic, Vertex AI, etc. so you can swap providers without rewriting call sites.
- PromptTemplate -- parameterized prompt strings with variable substitution.
- LCEL (Runnable) -- compose steps with the | pipe operator: prompt | llm | parser.
- Retriever -- standard interface wrapping any vector store behind .invoke(query).
- Memory -- classes like ConversationBufferMemory that auto-inject chat history.
- AgentExecutor -- an LLM in a loop that picks tools, observes results, decides to call another tool or stop (ReAct-style).

```
prompt = ChatPromptTemplate.from_template("Answer using {context}\n\nQ:\n{question}")
llm = ChatAnthropic(model="claude-3-5-sonnet-latest")
chain = prompt | llm # LCEL composition
response = chain.invoke({"context": docs, "question": q})
```

TIP: Value proposition: don't reinvent retrieval/prompt/memory plumbing -- swap vendors easily, get built-in tracing (LangSmith), lean on a large integration ecosystem.

2 What is LangGraph?

LangGraph is LangChain's sister framework for building stateful, cyclic, multi-step LLM workflows as an explicit graph: nodes (functions/LLM calls) and edges (control flow, including conditional branches and loops). Built because linear chains can't naturally express loops (retry until valid, keep calling tools until done).

Core building blocks:

- StateGraph -- a shared State (typically a TypedDict) flows through every node.
- add_node(name, fn) -- each node is a function (state) -> partial_state_update.
- add_edge(from, to) -- deterministic transition.
- add_conditional_edges(from, router_fn, mapping) -- branch based on a function's output (e.g. route by domain, or 'retry vs proceed').
- Cycles are first-class -- a node can route back to an earlier node (e.g. 'SQL failed -> back to generate_sql node with the error in state').
- Checkpointing/persistence -- pause, resume, human-in-the-loop, durable state.

```
class State(TypedDict): question: str; sql: str; error: str|None; retries: int

g = StateGraph(State)
g.add_node("generate_sql", generate_sql)
g.add_node("execute_sql", execute_sql)
g.add_edge("generate_sql", "execute_sql")
g.add_conditional_edges("execute_sql", route_after_execute,
{"generate_sql": "generate_sql", END: END}) # loop back on error
```

TIP: Value proposition: models workflows that are naturally graphs (agents, retries, human approval, multi-agent handoff) far more clearly than nested if/else, plus gives visualization, persistence, and streaming of intermediate state for free.

3 What Qcompass-Chatbot-KPI Actually Does

Verified directly from the codebase: no LangChain, no LangGraph, no agent framework anywhere. requirements.txt has no langchain/langgraph packages -- only raw SDKs: anthropic (Claude), google-cloud-aiplatform (Vertex embeddings), qdrant-client + fastembed (hybrid search), fastapi.

The actual flow -- UniversalPipeline.run():

```
UniversalPipeline.run():  
retrieve() -> _build_prompt() -> generate_sql() -> run_query()  
-> (on failure) _execute_with_retry [ONE hardcoded retry, try/except]  
-> generate_llm_dashboard()
```

api/main.py's /api/chat handler chains: get_recent_messages -> query_condenser.get_effective_question -> classify() -> run_domain_pipeline() -> background tasks (save_turn, user_memory). All plain function calls, no framework runtime.

- Routing (domain_classifier.py): hand-rolled hybrid scorer -- keyword hit-counting + embedding cosine similarity (weighted 0.4/0.6) + Claude LLM fallback below a confidence threshold.
- Registry (domain_registry.py): a plain dict of domain-name -> pipeline callable, built from domains_config.yaml.
- Memory (chat_memory.py): raw parameterized BigQuery DML, no LangChain Memory class.
- Tracing (tracer.py): local JSON-file writer gated by an env flag, no LangSmith.

INFO: query_condenser.py line 18 cites 'LangChain's history-aware retrieval' as design inspiration -- the team knew the pattern, but reimplemented it in plain Python rather than adding the dependency. No ADR/doc exists explicitly justifying skipping LangChain/LangGraph.

KEY IDEA: One-sentence summary: a modular monolith -- one UniversalPipeline class parameterized per-domain by YAML, run as a straight-line sequence of hand-written functions, with a dict-based router for domain dispatch.

4 Deep Dive: Dict-Based Routing as 'Graph Routing'

domain_registry.py builds a plain dict of domain-name -> pipeline callable from YAML config. This is functionally identical in BEHAVIOR to LangGraph's conditional-edge routing, but implemented as a dictionary lookup instead of a StateGraph object.

Step by step (actual code):

- At startup, domains_config.yaml is read once. For every domain (rcm_miu, jira_kpi, people_ops, ...), a closure is built: a tiny function that, when called, runs UniversalPipeline(domain_name).run(...) for that specific domain.
- Each closure is stored as a value in a plain dict, keyed by domain name:
DOMAIN_REGISTRY["rcm_miu"] = {"pipeline": <closure>, ...}.
- This dict is built ONCE at import time -- a module-level singleton for the process lifetime.
- At request time: classify(question) returns a plain string like "rcm_miu". run_domain_pipeline() does
config = get_domain_config(domain) -> config["pipeline"](question, ...) -- i.e. dict key lookup + calling the function stored there.

```
# domain_registry.py
for name, cfg in configs.items():
    def _make_pipeline(domain_name):
        def _pipeline(question, top_k=6, history=None, user_id=None):
            return UniversalPipeline(domain_name).run(question, top_k=top_k, ...)
        return _pipeline
    registry[name] = {"pipeline": _make_pipeline(name), ...}

# api/main.py
domain, method = classify(effective_question) # -> "rcm_miu"
result = run_domain_pipeline(domain, effective_question, ...)
# internally: DOMAIN_REGISTRY[domain]["pipeline"](question, ...)
```

Why this equals 'graph routing' in behavior:

```
# LangGraph equivalent
g.add_conditional_edges("classify", lambda state: state["domain"], {
    "rcm_miu": "rcm_miu_pipeline_node",
    "jira_kpi": "jira_kpi_pipeline_node",
    ...
})
```

KEY IDEA: The conditional-edge mapping (domain_name -> node_to_run) is structurally identical to DOMAIN_REGISTRY[domain_name] -> pipeline_callable. LangGraph wraps the exact same idea -- 'a string key decides which function runs next' -- inside a formal graph object that ALSO gives visualization, typed state, cycles, and checkpointing. The dict version has the same routing behavior with none of that extra scaffolding.

TIP: The behavior (dynamic dispatch to one of N branches based on a classifier's output) is equivalent between both approaches; only the implementation vehicle (raw dict + closures vs. a framework graph object) differs.

5 The Difference, Concretely

Aspect	Qcompass (hand-rolled)	If LangChain	If LangGraph
LLM calls	Direct anthropic SDK calls	ChatAnthropic wrapper + LCEL chains	Same wrapper, inside graph nodes
Retrieval	Direct qdrant_client.query_points()	Retriever interface wrapping Qdrant	Same, wrapped as a graph node
Routing (4 domains)	dict lookup + custom classifier	Not core focus (still hand-written)	add_conditional_edges + router fn
Retry-on-SQL-err or	One hardcoded try/except retry	Not natively supported (linear)	Conditional edge looping back, retries in State
Memory	Raw BigQuery DML	ConversationBufferMemory / history classes	State field carried through graph
Tracing	Local JSON file writer	LangSmith (hosted)	LangSmith (same)
Agentic behavior	None (single pass + 1 retry)	AgentExecutor (loop, less controllable)	Explicit, visualizable, controllable loop

6 Pros and Cons

Pros of the current hand-rolled approach:

- Zero framework overhead/version churn -- LangChain has a history of frequent breaking API changes; full ownership of code and stability.
- Full transparency/debuggability -- every step is a plain function, no framework 'magic' (hidden prompt injection, internal retry/parsing) to reverse-engineer.
- Smaller dependency footprint and attack surface -- fewer transitive packages.
- No abstraction-layer latency (Runnable wrapping, callback dispatch, tracing hooks).
- Exact control over prompt construction -- critical for consistent SQL generation.
- Tailored to a fixed-shape pipeline (classify -> condense -> retrieve -> SQL -> execute -> dashboard) that never actually varies per-request.

Cons of the current approach:

- Reinventing plumbing -- retry logic, memory, tracing all built from scratch instead of using battle-tested primitives.
- No built-in visualization/observability -- LangGraph gives a graph diagram and step-by-step state inspection for free; here it's custom logs only.
- Swapping LLM providers means touching call sites directly, not one wrapper config.
- Retry logic is a single hardcoded attempt -- growing requirements (retry N times, escalate to another model) means hand-rolling more nested control flow, exactly what LangGraph's cyclic graphs solve cleanly.
- Harder to add true agentic behavior later (e.g. LLM decides to re-query vs ask a clarifying question) without a significant rewrite.
- Less discoverable to new engineers -- LangChain/LangGraph patterns are widely documented; a custom pipeline requires reading the actual code.

CAUTION: Likely reasons for skipping the frameworks (inferred, no ADR exists): the pipeline is a fixed linear sequence per domain (not an open-ended agent), SQL generation is extremely prompt-sensitive (full prompt control reduces debugging variables), and the team wanted a small, dependency-light, self-hosted stack (local JSON tracing, raw BigQuery) -- consistent with the ONNX/FastEmbed 'avoid heavy dependencies' philosophy elsewhere in this stack.

7 Master Summary Table

Concept	One-line definition	Key mechanism
LangChain	Framework for LLM app plumbing (prompts, retrievers, memory)	LCEL: prompt llm parser
LangGraph	Framework for stateful, cyclic LLM workflows	StateGraph: add_node / add_edge / add_conditional_edges
UniversalPipeline	Qcompass's hand-written per-domain pipeline	Sequential calls: retrieve -> prompt -> SQL -> execute -> dashboard
Domain registry	dict mapping domain-name -> pipeline callable	Built once from YAML; behaves like graph routing without the graph
Hybrid classifier	Custom domain router	$0.4 * \text{keyword_score} + 0.6 * \text{embedding_cosine}$, + LLM fallback below threshold
<code>_execute_with_retry</code>	Hardcoded single retry on SQL failure	try/except ValueError, one extra attempt -- what LangGraph would model as a loop

8 Decision Guide

Scenario	Best approach	Why
Fixed, linear, well-understood pipeline shape per request	Hand-rolled functions	Framework overhead buys little when the shape never varies
Need to swap LLM/vector-store vendors frequently	LangChain	Unified wrapper interfaces avoid touching call sites
Genuine cycles/branches driven by runtime state (agent tool loops, retries, human approval)	LangGraph	Conditional edges + cycles are first-class; hand-rolling this gets messy fast
Need built-in tracing/visualization without extra infra	LangGraph + LangSmith	Graph diagram + state inspection out of the box
Want minimal dependencies, full prompt control, small footprint	Hand-rolled (Qcompass's choice)	Matches the project's 'avoid heavy deps' pattern seen elsewhere (ONNX/FastEmbed for embeddings too)

9 Interview Cheat Sheet (Q&A;)

Q. What's the core difference between LangChain and LangGraph?

A. LangChain = linear composition (chains/LCEL) + integrations (LLM wrappers, retrievers, memory).
LangGraph = explicit stateful graph with cycles/conditional routing, built to handle workflows LangChain's linear chains can't (loops, agent tool-calling, retries).

Q. Does Qcompass-Chatbot-KPI use LangChain or LangGraph?

A. No. Verified via requirements.txt and the codebase -- zero langchain/langgraph dependencies. All LLM calls (anthropic SDK), retrieval (qdrant-client), and orchestration are hand-written plain Python.

Q. How does domain routing work without LangGraph?

A. domain_classifier.py scores the question (keyword + embedding cosine, weighted 0.4/0.6, with an LLM fallback below a confidence threshold) and returns a domain name string. domain_registry.py has pre-built a dict mapping that string to a pipeline closure; run_domain_pipeline() looks it up and calls it.

Q. Why call dict-based routing 'functionally graph routing'?

A. Behaviorally, both do the same thing: given some input, dynamically dispatch to one of N functions/nodes based on a classifier's output. LangGraph formalizes this as add_conditional_edges(router_fn, {key: node}); the dict version is the same mapping without the graph object, visualization, or built-in state/cycle support.

Q. How is SQL-retry handled without a graph engine?

A. A single hardcoded try/except around run_query() -- on ValueError it retries generate_sql once with the error appended to context. In LangGraph this would be a conditional edge looping back to the generate_sql node, with a retries counter tracked in shared State, supporting N attempts instead of exactly one.

Q. What would you lose/gain by migrating this pipeline to LangGraph?

A. Gain: visualized graph, native N-attempt retry loops, easier agentic extensions, checkpointing/pausing.
Lose: full prompt-construction transparency, some latency overhead, an added dependency with its own versioning/breaking-change risk.

Q. What's a closure and why does domain_registry.py use one?

A. A closure is a function that captures variables from its enclosing scope. _make_pipeline(domain_name) returns a _pipeline function that 'remembers' domain_name even after _make_pipeline has returned -- needed because a plain loop variable would be shared/mutated across all dict entries otherwise (classic Python late-binding bug).

Q. What is LCEL in LangChain?

A. LangChain Expression Language -- lets you compose Runnables with the | operator, e.g. prompt | llm | output_parser, producing a single Runnable you can .invoke/.stream/.batch.

Q. Why might a team deliberately avoid LangChain/LangGraph for a production RAG system?

A. To avoid breaking-change churn across versions, keep full control over exact prompt text (critical when generating SQL), minimize dependency/attack surface, and avoid paying for or standing up LangSmith when simple local logging suffices.

Q. If asked to add human-in-the-loop approval before executing generated SQL, how would each approach handle it?

A. Hand-rolled: add another explicit pause/return step and a follow-up API call to resume -- more manual state management. LangGraph: add an 'awaiting_approval' node with native checkpointing to pause/resume graph execution across requests, which is exactly the scenario LangGraph's persistence layer was designed for.

10 One-Page Quick Reference

LangChain: prompt | llm | parser (linear composition + integrations)
LangGraph: StateGraph + add_node + add_conditional_edges (cycles, state, routing)

Qcompass pipeline (no framework):
classify() -> dict[domain]["pipeline"](question)
UniversalPipeline.run():
retrieve -> build_prompt -> generate_sql -> run_query
-> (on error) ONE retry -> generate_llm_dashboard

Dict routing == graph routing (behaviorally):
DOMAIN_REGISTRY[domain] -> pipeline_callable
~= add_conditional_edges(router_fn, {domain: node})
Same dispatch behavior; graph adds visualization + state + cycles + checkpointing.

KEY IDEA: Golden rule: frameworks buy you cycles, state, and observability for free -- but cost you dependency churn and reduced prompt-level control. Pick based on whether your workflow's SHAPE actually varies at runtime.